

EagleEye: Balancing Latency, Accuracy, and Power on Edge-Assisted UAVs for Urban Crowd Surveillance

Chaocan Xiang¹, Zhengnan Li, Qianyuan Zhang, Xuanguo Wu², Xiao Zheng³,
and Yulan Guo⁴, *Senior Member, IEEE*

Abstract—Uncrewed Aerial Vehicles (UAVs) equipped with cameras provide a promising way for large-scale urban crowd surveillance due to their convenient deployment and flexible mobility. However, UAVs are constrained by limited power and computing resources, hindering existing work in achieving efficient UAV-based crowd surveillance, i.e., long flight time, high accuracy, and low latency. To this end, we propose *EagleEye*, a low-power, high-precision, and low-latency crowd surveillance system empowered by edge-assisted UAVs. It leverages lightweight devices on UAV-side to compress video information edge-independently, then transmits key video information instead of raw high-definition videos. Furthermore, we propose a novel spatio-temporal Compressive-Sensing-based video feature compression algorithm to achieve efficient, low-latency video compression. It can reduce video volumes greatly while minimizing the loss of crowd-surveillance-related information. Specifically, inspired by the Compressive Sensing theory, we compress the video content from both the temporal and spatial perspectives by accounting for inter-frame redundancy and intra-frame information saliency, respectively. Finally, we implement a prototype system and conduct extensive experiments based on four large-scale datasets with over 20,000 frames. The experimental results demonstrate that *EagleEye* can reduce transmission latency by 31.4% only with no more than 4% of accuracy loss in urban crowd detection.

Index Terms—Uncrewed Aerial Vehicles (UAVs), Urban crowd surveillance, edge computing, video feature compression.

I. INTRODUCTION

IN RECENT years, the rapid development of urbanization has led to a significant surge in population density in certain cities, thereby increasing the risk of stampedes and riots, and posing numerous threats to public safety [2]. For example, over

2,000 people died during the Mina stampede, which occurred in Saudi Arabia in September 2015 [3]. To effectively mitigate such hazards, crowd surveillance plays a vital role in urban emergency management. The critical applications of crowd surveillance systems are violence detection [4], facial recognition [5], and crowd counting [6]. Violence detection can identify conflicts and dangers in real-time, providing timely information for emergency responses; facial recognition is important for criminal investigations and locating missing persons; crowd counting analyzes crowd density and distribution, assisting in planning public facilities and emergency evacuation routes. Traditional methods of crowd surveillance typically rely on fixed cameras [7], security personnel patrols [8], and various sensors (e.g., millimeter-wave radar) [9]. However, due to limited detection ranges, regular maintenance requirements, and reliance on human resources, these methods often suffer from insufficient coverage [10], high costs, and poor adaptability to environmental changes.

Uncrewed Aerial Vehicles (UAVs) have become popular in both civilian and commercial applications, such as aerial photography [11], sensor data collection [12], and emergency response [13]. Their flexible mobility and wide-range video-capturing capabilities give camera-equipped UAVs a promising approach for large-scale urban crowd surveillance [6]. As a result, many studies [14], [15], [16], [17], [18], [19], [20], [21] have been proposed to realize UAV-based crowd surveillance. However, most works [14], [15], [16], [17] focus on high-precision surveillance, overlooking the significant latency caused by high-definition video transmission. For example, as shown in Fig. 2(a), transmitting an 800p, 10-minute video from a UAV to a Ground Control Station (GCS), located only 30 meters away, results in a latency of over 30 minutes, severely hindering real-time surveillance. To reduce the delay of transmitting high-definition videos to the server, some studies [4], [18], [19] use simple video compression methods to compress the videos. However, this approach leads to significant video quality loss, thereby reducing detection accuracy noticeably. For example, as shown in Fig. 2(b), compressing video frames from 800p to 320p drops the detection accuracy from 80% to 64%.

To address this issue, many studies [22], [23] have explored using high-performance edge devices for executing complex video compression algorithms to reduce the transmitting latency. However, they overlooked the fact that high-performance

Received 12 January 2025; revised 24 June 2025; accepted 3 July 2025. Date of publication 8 July 2025; date of current version 5 November 2025. This work was supported in part by NSF China under Grant 62172063 and Grant 62441207, and in part by NSF of Chongqing Projects under Grant CSTB2024NSCQ-JQX0020. Recommended for acceptance by C.H. Liu. (Corresponding author: Yulan Guo.)

Chaocan Xiang, Zhengnan Li, and Qianyuan Zhang are with the College of Computer Science, Chongqing University, Chongqing 400044, China.

Xuanguo Wu and Xiao Zheng are with the School of Computer Science and Technology, University of Anhui Technology, Maanshan 243002, China.

Yulan Guo is with the School of Electronics and Communication Engineering, Shenzhen Campus of Sun Yat-sen University, Sun Yat-sen University, Shenzhen 518107, China (e-mail: guoyulan@sysu.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TMC.2025.3586860>, provided by the authors.

Digital Object Identifier 10.1109/TMC.2025.3586860

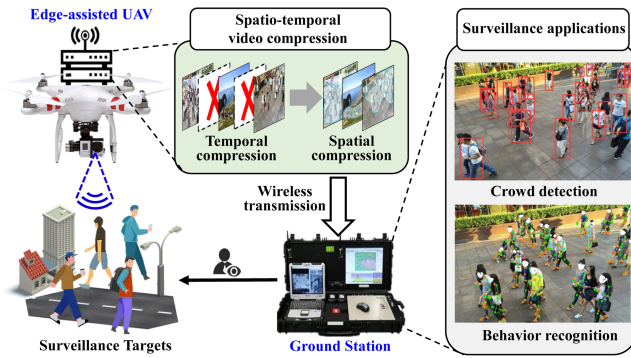


Fig. 1. Framework of *EagleEye*, a real-time crowd surveillance system empowered by edge-assisted UAVs.

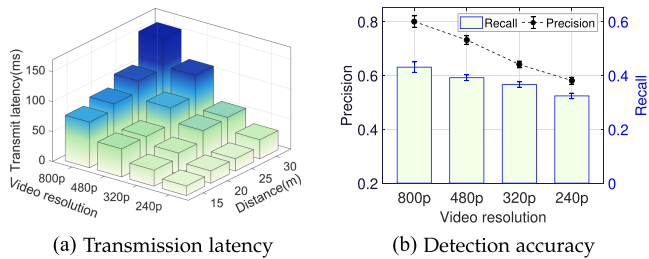


Fig. 2. Real experiments of UAV-based surveillance for exploring the latency-accuracy dilemma.

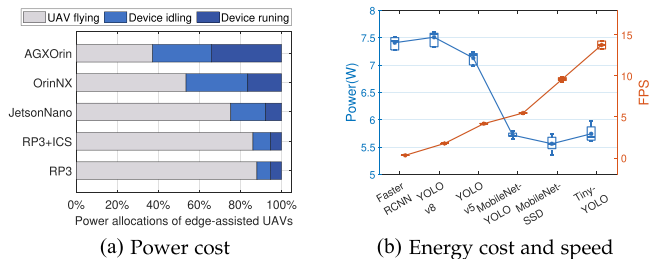


Fig. 3. Experiments on power cost (a) and running speed of edge-side computing devices and models on Jetson Nano (b).

devices typically consume more power, thereby reducing the flight endurance of UAVs. For example, as shown in Fig. 3(a), deploying edge devices on UAVs increases power consumption, and high-performance devices can even reduce the flight range by over 60%. Similarly, as shown in Fig. 3(b), using more accurate object detection models on the Jetson Nano also increases the power consumption and reduces the processing speed of videos. Therefore, it increases the power consumption of UAVs and the data processing delays, which cannot be applied to real-time urban crowd surveillance systems. To mitigate power consumption, references [24] have employed low-power video compression and lightweight devices, primarily relying on cloud servers to assist edge devices with data feedback or model updates. However, edge devices rely on data feedback from the GCS or the cloud, which consumes video transmission bandwidth, resulting in significant delays in data transmission and processing. Furthermore, these approaches compress videos in either the temporal or spatial dimensions,

leading to spatio-temporal redundancy. In summary, *the existing works cannot achieve a balance between power consumption, detection latency, and detection accuracy, incurring significant obstacles to their practical applications.*

EagleEye is proposed as a solution to the above problems, which is a real-time crowd surveillance system supported by edge-assisted UAVs. Specifically, as shown in Fig. 1, we deploy lightweight devices on the UAV side to independently compress redundant frames in the drone-captured videos. These processed frames are transmitted to the GCS instead of the raw high-definition videos. After receiving the processed frames, the GCS executes crowd surveillance algorithms on a high-performance cloud server to perform real-time analysis of crowd density, violent behavior, and target identification, providing support for urban public safety management. As a result, it achieves accurate crowd surveillance with low latency under low-power conditions without requiring data feedback from the GCS or the cloud. However, realizing this system is non-trivial with the following two challenges:

- *Complex keyframe selection for removing temporal redundancy:* We use lightweight devices to meet our low power consumption needs. However, these devices have limited processing capabilities, which makes it challenging to eliminate temporal redundancy by extracting keyframes from videos. Additionally, the complex relationship between crowd count and frame similarity makes keyframe extraction even more difficult, thereby increasing the challenge of removing temporal redundancy from video frames.
- *Accurate RoI identification for removing spatial redundancy:* Accurately identifying the region of interest (RoI) within video frames is essential for adaptive video compression, as it helps eliminate spatial redundancy. However, the system relies on lightweight devices that can only support lightweight models. Unfortunately, these models may lead to inaccurate crowd detection, resulting in improper RoI partitioning and ineffective removal of spatial redundancy. Therefore, the challenge lies in accurately identifying the RoI to eliminate spatial redundancy with limited processing power.

To address these challenges, we propose a *novel video feature compression framework empowered by spatio-temporal compressive sensing*, which is tailored to UAV-side edge devices with limited computational capabilities. Specifically, it consists of two key components: 1) *Compressive-sensing-based temporal video feature compression*. We investigate the low-rank property of video feature matrices via Singular Value Decomposition (SVD) analysis. Based on the results, we exploit the compressive sensing theory for video feature compression to retain keyframes accurately and minimize inter-frame redundancy. 2) *Dynamic-encoding-based spatial video content compression*. Leveraging edge detection and region proposal partitioning, we dynamically identify the intra-frame information saliency and compress videos from a spatial aspect. At the same time, we minimize the intra-frame non-saliency data while maintaining the visual quality of crowd regions.

In summary, we make three key contributions:

- 1) We present an intelligent UAV-based edge computing system called *EagleEye*, which efficiently utilizes the limited capability of UAV-side edge devices to *achieve latency-accuracy-power balance* for crowd surveillance.
- 2) We design a lightweight spatio-temporal compressive-sensing-based video feature compression framework, tailored for the limited computing capability of UAV-side edge devices. Its basic idea is to utilize Compressive Sensing (CS) theory and information-saliency-based dynamic encoding to compress video from both temporal and spatial perspectives. Hence, it minimizes video transmission latency with minimal loss of accuracy in crowd surveillance.
- 3) We build the system prototype and conduct extensive performance evaluations. The experimental results show that *EagleEye* reduces the transmission delay by 31.4% and decreases the power consumption by 7.2% while sacrificing no more than 4% of the crowd-surveillance accuracy, hence achieving a good latency-accuracy-power trade-off.

The rest of the paper is organized as follows. First, we review related work in Section II and present the system architecture in Section III. Then, Section IV describes our spatio-temporal compressive-sensing-empowered video compression algorithm. Moreover, we conduct extensive experiments and implement the real-world system prototype in Section V. Next, we make discussions in Section VI. Finally, we conclude this paper in Section VII.

II. RELATED WORK

Drones have been widely utilized across various applications, such as crowd surveillance [17], [25], [27] and emergency response [33], due to their unique advantages of rapid deployment and flexible mobility. Since this paper falls within the category of UAV-based crowd surveillance, we provide a comprehensive review of related work, categorizing them into three types: (1) *traditional drone-based urban crowd surveillance without video compression*, (2) *video compression in drone scenarios*, and (3) *video compression in other scenarios*.

Drone-based crowd surveillance without video compression: As one of the earliest works about traditional surveillance, Rezaee et al. [17] employ ScatterNet for human pose estimation to identify violent behavior from UAV-captured footage. Considering the efficiency of surveillance coverage, MahmoudZadeh et al. [25] propose a distributed UAV surveillance algorithm without guaranteed accuracy. In addition, to improve accuracy in adverse environments, Deng et al. [33] use drones equipped with millimeter-wave radar to compensate for the lack of visual information, achieving accurate surveillance results. Castellano et al. [26] propose a fully convolutional network method based on group clustering, which can perform efficient crowd surveillance on drone video sequences. However, since this work uses complex network models, it causes faster power consumption for drones. In summary, *these traditional works require transmitting raw videos to ground stations or cloud servers, which results in significant transmission delays to hinder their applicability in time-critical emergencies.*

Video compression in drone scenarios: To reduce the transmission time of raw video captured by drones to the GCS, most UAV-based systems perform video compression or keyframe extraction directly on the drone and then transmit the processed video to the server for further analysis. For example, Guo et al. [27] propose a real-time drone-assisted video analysis technique, built on existing codecs and using drone flight trajectories and object movement to predict frames and reduce video encoding volume. Moreover, Xu et al. [28] propose a lightweight drone-based human recognition model. It removes the redundant frame and offloads data from the drone to the ground station, reducing the video transmission bandwidth. Aakanksha et al. [29] propose a model predictive compression algorithm to predict drone trajectories and transmit key image frames to the GCS, hence minimizing network bandwidth and achieving high-quality video stitching. However, these works do not consider power consumption. Zhang et al. [30] present a computation offloading algorithm based on deep deterministic policy gradient to improve the accuracy of video detection tasks and reduce drone power consumption. Nevertheless, their work does not consider the data offloading delays. To sum up, *although these studies employ various video compression algorithms to process videos captured by drones for low-bandwidth video transmission, it is challenging to balance power consumption, latency, and accuracy simultaneously. This limitation affects the effectiveness of drones in the urban crowd surveillance system. Therefore, new methods need to be explored to achieve a balance among these three aspects.*

Video compression in other scenarios: To explore a wide range of video compression technologies, we investigate video compression in other scenarios. Jiang et al. [21] partition each frame adaptively; different partitions are processed by different networks, achieving high accuracy and fast inference speed. Nevertheless, it uses high-performance devices to execute complex algorithms, thereby incurring excessive power consumption. Li et al. [31] perform feature extraction and frame filtering on lightweight devices, thus reducing bandwidth consumption between edge devices and servers while achieving high-precision object recognition. However, this work does not consider compressing each frame on the device, resulting in too much memory consumption. Liu et al. [20] achieves high-precision and low-latency object detection in AR/MR systems through low-latency offloading technology. However, in this work, the AR device relies on feedback from the server to compress future frames, which takes up the bandwidth of video transmission and reduces the video transmission speed. In summary, *it is not necessary for other scenarios to consider the constraints of UAV systems when applying the video compression algorithms. In most cases, these works employ complex algorithms to achieve greater video memory reduction and minimize video quality loss. However, they often ignore the selection of edge devices, the complexity of compression algorithms, and power consumption issues, which are crucial for drone scenarios.*

Summary: As shown in Table I and Fig. 4, on the one hand, existing works fail to maintain low-loss data while achieving low power consumption and low latency in drone scenarios. On the other hand, video compression works in other scenarios are often more complex and advanced, overlooking power consumption

TABLE I
COMPARISONS BETWEEN EXISTING WORKS AND *EAGLEEYE* REGARDING THE SYSTEM MODEL AND ALGORITHM PERFORMANCE

	Drone scenarios without video compression		Video compression in drone scenarios				Video compression in other scenarios		Ours
	[7], [25]	[17], [26]	[27]	[28]	[29]	[30]	[20], [31]	[21], [32]	
Low power cost	×	×	×	×	×	✓	×	×	✓
Low transmission latency	×	×	✓	×	✓	×	✓	×	✓
Low process latency	×	✓	×	✓	✓	✓	✓	✓	✓
High detection accuracy	×	✓	✓	✓	×	✓	✓	✓	✓

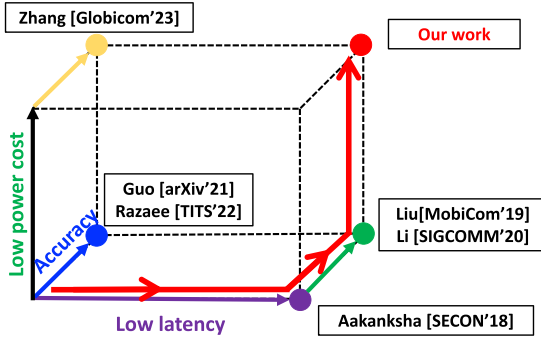


Fig. 4. Comparisons between existing works and *EagleEye*.

issues related to edge devices and compression algorithms. Distinct from these existing works, *we propose a real-time crowd surveillance system empowered by edge-assisted UAVs, leveraging a novel spatio-temporal video compression method to achieve a good balance between power consumption, latency, and accuracy.*

III. OPPORTUNITIES & CHALLENGES IN SYSTEM DESIGN

System Overview: We present an overview of the UAV-assisted edge computing system called *EagleEye* for real-time crowd surveillance. As shown in Fig. 1, it consists of two components:

- *Edge-assisted UAV sensing module:* This module consists of drones and mounted lightweight edge devices, such as Jetson Nano [34]. First, it performs mobile surveillance and video capturing of crowd areas. Then, the edge device compressed the captured videos and transmitted them to the GCS in real time.
- *Ground station based analysis module:* This module comprises a GCS and a central server for crowd surveillance. It can manage the UAV's flight trajectory and analyze the compressed videos from the UAVs for crowd surveillance applications.

Note that we do not perform crowd surveillance tasks directly on the edge-assisted UAV sensing module, primarily accounting for the system's power consumption and latency as follows: (1) The limited power and payload capacity of the UAV cannot support the long-term computation of the model on the edge device. Computation tasks on the edge device increase the UAV's power consumption and affect its flight duration. To ensure a longer flight duration, the optimal solution is to compress the video on the sensing module and then upload it

to the GCS for processing. (2) Although the performance of lightweight embedded devices has been improved, there is still a significant performance gap compared to cloud or local servers. These gaps render the device incapable of meeting the real-time requirements of complex crowd surveillance applications, such as violence detection, crowd re-identification, and crowd image segmentation. Running these algorithms on edge devices may lead to high delays and fail to achieve the same performance as the GCS.

Opportunity Exploration: The goal of *EagleEye* is to achieve accurate crowd surveillance with low latency. However, the observations in Fig. 2(a) and (b) indicate that these two goals cannot be satisfied simultaneously, and existing works have not addressed this issue effectively. Thus, to address this problem, we propose a promising approach to perform efficient video compression for edge-assisted drones. This approach is attributed to the opportunities for video compression from temporal and spatial perspectives as follows:

- 1) *Opportunities for temporal video compression:* We compute the pixel-wise frame similarity between two sequential video frames. As shown in Fig. 5(a), the crowd counts and the frame similarity are positively correlated. Therefore, we can filter the video frames from the temporal aspect according to frame similarity.
- 2) *Opportunities for spatial video compression:* We analyze the proportions of background and non-background pixels in drone-captured videos. One example is illustrated in Fig. 5(b), which shows that background regions occupy 67.4% of the image area. Thus, compressing the background regions within each frame can substantially decrease the transmission latency.

Challenge Analysis: Based on the above two opportunities, our basic idea of video compression is to *extract key video features in crowd surveillance tasks while removing redundant content from temporal and spatial aspects, hence achieving low-latency surveillance while maintaining high detection accuracy.* Nevertheless, three challenges should be addressed:

- 1) *Extreme constraint in processing capability:* As shown in Fig. 3(a), we survey the most common commercial edge devices. If a regular UAV (e.g., S500 [35]) is equipped with a high-performance device (e.g., Nvidia AGX Orin [36]) with 1.5 kg and 40 W, the endurance is reduced by more than 60%. Furthermore, we compare the performance and compatibility of these edge devices (the details are in the supplementary material, available online). Lightweight devices (e.g., Raspberry Pi3) typically offer lower performance and poorer compatibility, rendering them

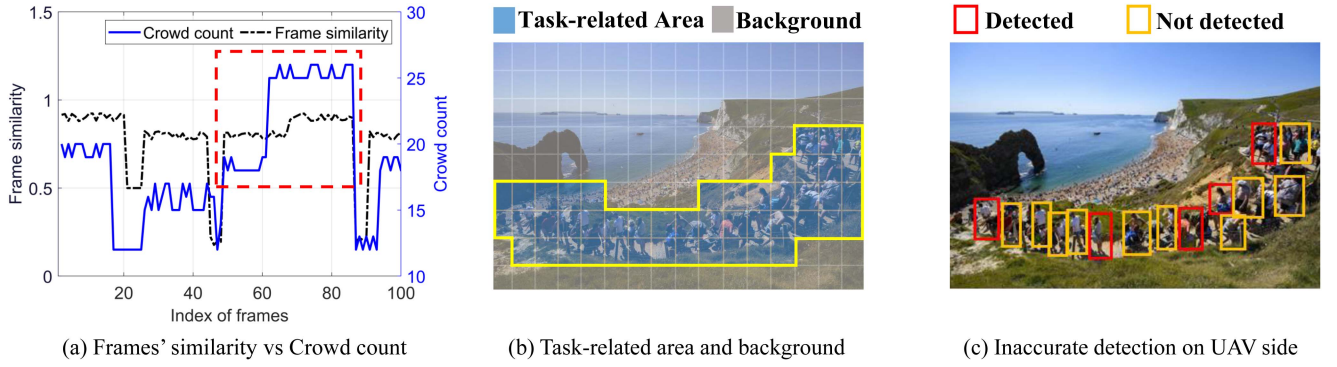


Fig. 5. Opportunity and challenge analysis for video compression, including the opportunities for (a) temporal video compression and (b) spatial video compression, as well as (c) the challenge in intra-frame saliency information extraction.

insufficient for running deep learning algorithms. Consequently, we are constrained to using lightweight devices (e.g., JetsonNano) that consume less power but provide limited AI performance and compatibility. It is greatly challenging to design an effective video compression algorithm under these processing constraints.

- 2) *Complexity of inter-frame video compression*: The relationship between the crowd count and the frame similarity is intricate. For example, Fig. 5(a) shows that from the 50th to 90th frames, although the corresponding crowd counts vary greatly, the inter-frame similarity remains almost unchanged, which means that directly removing redundant frames based on inter-frame similarity is highly inaccurate. Furthermore, since UAV has limited processing capability, compressing inter-frame redundancy effectively is highly challenging.
- 3) *Difficulty of intra-frame saliency information extraction*: We attempt to utilize MobileNet-yolo [37] to detect crowds in UAV-captured frames on edge devices. Fig. 5(c) shows that 66.7% of the crowds are not detected due to the lightweight detection model. Therefore, using lightweight detection models directly leads to misidentifying salient information within the frames. It is necessary to use additional methods to improve the accuracy of extracting salient information from the frames.

IV. ALGORITHM DESIGN

In this section, we propose a novel *spatio-temporal compressive sensing-based video feature compression algorithm* for UAV-assisted crowd surveillance. This algorithm addresses video transmission delay issues and the limited processing capability of edge devices. It consists of two key components:

- *Temporal Video Feature Compression Leveraging Compressive Sensing (Section IV-A)*: Leveraging the Compressive Sensing theory with both the crowd feature and the environment profile feature, we extract useful features of keyframes and remove the redundant frames of UAV-captured videos.
- *Spatial Video Content Compression by Dynamic Encoding (Section IV-B)*: We identify the encoding RoIs of the

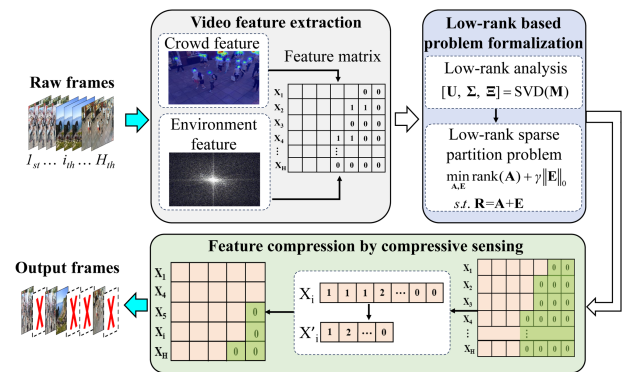


Fig. 6. Illustration of temporal video feature compression, including video feature extraction, problem formalization via low-rank analysis, and CS-based video feature compression.

frames and dynamically encode each frame according to the RoI divisions, thereby further reducing the inefficient information inside a frame.

A. Temporal Video Feature Compression Leveraging Compressive Sensing

In this subsection, we remove the redundant frames from the temporal aspect. Specifically, as shown in Fig. 6, we first extract the *crowd feature* and the *environment profile feature* of video frames, which are used to identify the redundant frames effectively by leveraging Compressive Sensing theory. Note that the goal of *EagleEye* is to achieve efficient crowd surveillance. Hence, it is not necessary to ensure the continuity of video frames. As a result, removing redundant frames can effectively reduce video transmission latency.

1) *Video Feature Extraction*: Inspired by feature compression technologies [38] in surveillance footage, we consider segmenting the effective and ineffective parts of the video frame on the UAV side to reduce the file volumes. Specifically, we first convert the UAV-captured frames into a comprehensive feature representation using the following feature extraction method.

- 1) *Crowd Feature*: The crowd feature represents the density and location distribution of the crowd in each frame,

TABLE II
 FREQUENTLY USED NOTATIONS

Notations	Descriptions
I_k	The k -th frame.
N_k	The number of bounding box of k -th frame.
d_i^k	The coordinate of i -th bounding box in k -th frame.
c_i^k	The confidence of i -th bounding box in k -th frame.
$\mathbf{m}_k$	The crowd density map of k -th frame.
$\mathbf{P}_k$	The crowd feature of k -th frame.
$\mathbf{S}_k$	The environment profile feature of k -th frame.
$\mathcal{V}_{orig}, \mathcal{V}_{out}$	The input and output of temporal compression.
$\mathcal{I}_{orig}, \mathcal{I}_{out}$	The input and output of spatial compression.
$\mathcal{R}_e$	The output RoIs of edge detection.
$\mathcal{R}_p$	The output batches of RP partition.
$\mathcal{R}_f$	The revised RoIs.
Θ_k	The information saliency map of k -th frame.
$\phi_k(i, j)$	The encoding quality of pixel (i, j) in k -th frame.

denoted as $\mathbf{P}_k$. Considering the detection speed and accuracy constraints of UAV-side edge devices, we employ a lightweight network model (e.g., Tiny YOLO [39]) to extract the crowd features. Specifically, the central coordinates and confidence levels of the i th bounding box in the frame are denoted by d_i^k and c_i^k ($c_i^k \in (0, 1]$), respectively. Let $\mathbf{m}_k$ denote the crowd density map. We convolve the detection results with a Gaussian kernel to obtain $\mathbf{m}_k$ as

$$\mathbf{m}_k = \sum_{i=1}^{N_k} c_i^k \cdot \delta(x - d_i^k) * G_{\delta_i}(x), \quad (1)$$

where N_k is the number of bounding boxes in the k th frame; $G_{\delta_i}(x)$ denotes a two-dimensional Gaussian kernel; $\delta(x - d_i^k)$ represents the impulse function. Thus, $\mathbf{m}_k$ is uniformly pooled to obtain the compressed crowd density map as the crowd features $\mathbf{P}_k$. The frequently used notations are listed in Table II.

- 2) *Environment Profile Feature*: The environment profile feature characterizes the low-frequency edge texture, luminance, and color distribution of an image, denoted as $\mathbf{S}_k$. First, we employ the 2-dimensional Discrete Cosine Transform (2D-DCT) [40] to retain the low-frequency region, where the spectral energy is concentrated as the frequency-domain feature. Then, we concatenate the grayscale variance and mean values of all patch blocks as the luminance and contrast features. Furthermore, the pixel distribution ratios of different grayscale levels for all color channels are computed as the color distribution features. Finally, the frequency-domain, luminance, contrast, and color distribution features are combined as the environment profile feature $\mathbf{S}_k$.

2) *Problem Formalization Based on Low-Rank Analysis*: For the frame sequence $\mathcal{V} = \{I_1, I_2, \dots, I_T\}$, its feature tensor can be denoted as $\{\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_T\}$, where $\mathbf{X}_k$ with H_k elements is combined of the crowd feature $\mathbf{P}_k$ and the environment profile feature $\mathbf{S}_k$. The feature vectors are combined to form the feature matrix of the sequential frames $\mathbf{R} \in \mathbb{R}^{T \times H}$ as

$$\mathbf{R} = \begin{pmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \vdots \\ \mathbf{X}_T \end{pmatrix} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,H} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,H} \\ \vdots & \vdots & \ddots & \vdots \\ x_{T,1} & x_{T,2} & \cdots & x_{T,H} \end{bmatrix}, \quad (2)$$

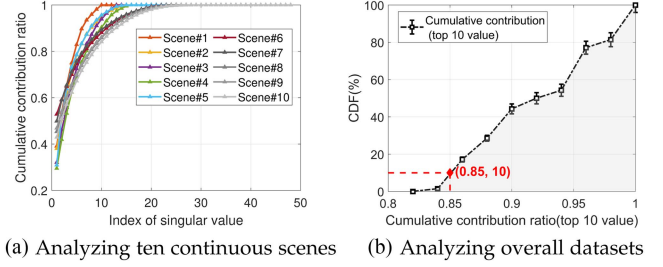


Fig. 7. Low-rank analysis of video feature matrices based on Singular Value Decomposition.

where $H = \max\{H_1, \dots, H_T\}$. If the length of each row is less than the maximum row length, it is padded with zero.

We first investigate whether the feature matrices of UAV-captured frames are low-rank by applying Singular Value Decomposition (SVD) [41]. Specifically, we conduct the SVD experiments on the feature matrices at varying scenes. As shown in Fig. 7(a), the top ten largest singular values contribute to over 85% of the sum of all singular values in these scenes. Moreover, Fig. 7(b) reveals that the cumulative contribution from the first ten largest singular values of the feature matrix exceeds 85% in most continuous scenarios (about 90%). According to the theorem of matrix rank [41], a matrix is approximately low-rank if the weights of its singular values concentrate on very few elements. Therefore, the feature matrices in surveillance footage are roughly low-rank, which inspires the extraction of efficient portions from UAV-captured videos.

Based on the low-rank matrix partitioning theorem [38], [42], [43], [44], [45], if the feature matrix $\mathbf{R}$ is approximately low-rank, it can be decomposed into a low-rank part $\mathbf{A}$ (i.e., redundant data) and a sparse part $\mathbf{E}$ (i.e., efficient information). Thus, the matrix low-rank sparse decomposition problem can be formalized as

$$\min_{\mathbf{A}, \mathbf{E}} \text{rank}(\mathbf{A}) + \gamma \|\mathbf{E}\|_0, \quad (3)$$

$$\text{s.t. } \mathbf{R} = \mathbf{A} + \mathbf{E}, \quad (4)$$

where $\text{rank}(\cdot)$ denotes the function of computing the matrix rank, i.e., the number of non-zero singular values; the ℓ_0 -norm $\|\cdot\|_0$ is used to measure the sparsity of a matrix; γ denotes the weight factor.

3) *Compressive Sensing Based Video Feature Compression*: Due to the non-convex and non-smooth properties of the $\text{rank}(\cdot)$ and ℓ_0 -norm, the matrix decomposition problem in (3)–(4) is NP-hard. Thus, it can be equivalently relaxed by substituting the nuclear norm $\|\cdot\|_*$ and ℓ_1 -norm $\|\cdot\|_1$ for the $\text{rank}(\cdot)$ and $\|\cdot\|_0$, respectively. Hence, we transform the low-rank sparse decomposition problem, an intractable NP-hard one, into a tractable convex optimization problem.

To further address this problem, Augmented Lagrange Multiplier (IALM) [43] based feature compression algorithm is exploited to achieve the optimal solution via limited iterations. By applying the augmented Lagrangian function to this problem, we can reformulate it as

$$(\mathbf{A}_{k+1}^*, \mathbf{E}_{k+1}^*) = \arg \min_{\mathbf{A}, \mathbf{E}} L(\mathbf{A}, \mathbf{E}, \mathbf{Y}_k, u_k), \quad (5)$$

Algorithm 1: Spatio-Temporal Compressive Sensing Based Video Feature Compression.

Input: UAV-captured continuous video sequence with H frames $\mathcal{V}_{orig} = \{I_1, \dots, I_H\}$
Output: Sampled video sequence with K frames $\mathcal{V}_{out} = \{I_i, \dots, I_j\}, 1 \leq i, j \leq H$.

```

1 Initialize:  $\mathcal{V}_{out} = \emptyset$ ; sliding step  $t$ ; edge detection threshold  $\varepsilon_d$ .
2 /* Temporal compression */
3 for  $i \leftarrow 0$  to  $\lfloor (H-t)/t \rfloor - 1$  do
4    $\mathbf{R} = \text{FeatureExtract}(\{I_{i \times t + 1}, \dots, I_{(i+1) \times t + t}\})$ 
5   set  $\mathbf{Y}_0 = \mathbf{R}/J(\mathbf{R}); \mathbf{E}_0 = 0; \mu_0 > 0; \rho > 1; k = 0$ 
6   while not converged do
7     Given  $\mathbf{E}$ , update  $\mathbf{A}_{k+1}$  by Eq. (6)
8     Given  $\mathbf{A}$ , update  $\mathbf{E}_{k+1}$  by Eq. (8)
9      $\mathbf{Y}_{k+1} = \mathbf{Y}_k + u_k(\mathbf{R} - \mathbf{A}_{k+1}^* - \mathbf{E}_{k+1}^*)$ 
10     $\mu_{k+1} = \rho \mu_k$ 
11     $k = k + 1$ 
12  $\mathcal{V}_{keep} = \text{MapToSeq}(\mathbf{E})$ 
13 /* Spatial compression */
14 for  $I$  in  $\mathcal{V}_{keep}$  do
15    $\mathcal{B} = \text{Division}(I)$ 
16   set  $\mathcal{R}_f = \emptyset; \mathcal{R}_e = \emptyset; \mathcal{R}_p = \emptyset$ 
17   for  $box$  in  $\mathcal{B}$  do
18      $\mathcal{R}_e = \text{EdgeDetect}(box, \varepsilon_d)$ 
19      $\mathcal{R}_p = \text{RPPartition}(box)$ 
20   if  $|\mathcal{R}_e| > 0$  or  $|\mathcal{R}_p| > 0$  then
21      $\mathcal{R}_f = \mathcal{R}_f \cup \{\mathcal{R}_e \cup \mathcal{R}_p\}$ 
22     /* lossless encoding */
23      $\Theta_x = \text{InfoSaliencyMap}(I, \mathcal{R}_f, \mathcal{R}_e, \mathcal{R}_p)$ 
24      $I = \text{LosslessEnc}(I, \Theta_x)$ 
25   else
26     /* lossy encoding */
27      $I = \text{LossyEnc}(I)$ 
28  $\mathcal{V}_{out} = \mathcal{V}_{out} \cup \mathcal{V}_{keep}$ 
29 return  $\mathcal{V}_{out}$ .
```

where $L(\mathbf{A}, \mathbf{E}, \mathbf{Y}, u)$ is equivalent to $\|\mathbf{A}\|_* + \lambda \|\mathbf{E}\|_1 + \langle \mathbf{Y}, \mathbf{R} - \mathbf{A} - \mathbf{E} \rangle + \frac{u}{2} \|\mathbf{R} - \mathbf{A} - \mathbf{E}\|_F^2$, and $\langle \mathbf{A}, \mathbf{B} \rangle$ denotes the trace of $\mathbf{A}^T \mathbf{B}$. As shown in Algorithm 1, our approach is comprised of two key steps to solve the optimization problem:

- *Singular Value Shrinkage* (line 7): Given the sparse part $\mathbf{E}$, we update $\mathbf{A}$ by the following updating scheme:

$$\mathbf{A}_{k+1} = \mathcal{D}_{\frac{1}{u_k}} \left(\mathbf{R} - \mathbf{E}_{k+1} + \frac{\mathbf{Y}_k}{u_k} \right), \quad (6)$$

where $\mathcal{D}(\cdot)$ is Singular Value Shrinkage Operator (SVSO) defined as $\mathcal{D}_\tau(\mathbf{M}) = \mathbf{U} \mathcal{S}_\tau[\mathbf{\Sigma}] \mathbf{\Xi}$. Note that $\mathbf{U}$, $\mathbf{\Sigma}$, and $\mathbf{\Xi}$ are generated from the SVD of matrix $\mathbf{M}$ with rank r , i.e., $[\mathbf{U}, \mathbf{\Sigma}, \mathbf{\Xi}] = \text{SVD}(\mathbf{M})$, $\mathbf{\Sigma} = \text{diag}(\{\sigma_i\}_{1 \leq i \leq r})$; $\mathcal{S}_\tau(\cdot)$ is the soft-thresholding operator as

$$\mathcal{S}_\tau(x) = \begin{cases} x - \tau \cdot \text{sign}(x), & \text{if } |x| > \tau, \\ 0, & \text{otherwise,} \end{cases} \quad (7)$$

where $\tau > 0$. This operator can be extended to vectors and matrices by applying it element-wise.

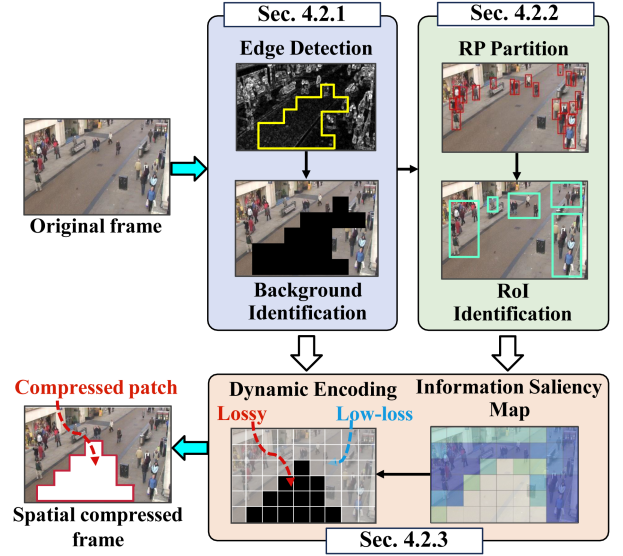


Fig. 8. Illustration of spatial video content compression, where edge detection and region proposal (RP) partition are implemented to identify task-related RoIs, followed by dynamic image encoding based on the information saliency map.

- *Soft-Thresholding* (line 8): Given the low-rank part $\mathbf{A}$, the sparse part $\mathbf{E}$ is updated by the following scheme:

$$\mathbf{E}_{k+1} = \mathcal{S}_{\frac{\lambda}{u_k}} \left(\mathbf{R} - \mathbf{A}_{k+1} + \frac{\mathbf{Y}_k}{u_k} \right). \quad (8)$$

The above two steps iterate with each other. At the end of each iteration, the parameters are updated as $\mathbf{Y}_{k+1} = \mathbf{Y}_k + u_k(\mathbf{R} - \mathbf{A}_{k+1}^* - \mathbf{E}_{k+1}^*)$. The iteration ends until one of the following stopping criteria is met: (1) the maximum number of iterations is reached; (2) the difference of the matrices $\mathbf{A}$ and $\mathbf{E}$ obtained in the $k+1$ -th and k -th iteration is not greater than the threshold ε_0 .

B. Spatial Video Compression Empowered by Dynamic Encoding

Although the temporal compression in Section IV-A has removed inter-frame duplicate content, a large amount of unnecessary background (e.g., streets, trees, and sky) still remains within the frames. Transmitting the original images to the GCS results in high transmission latency. Therefore, we should reduce the memory size of each frame to achieve low-latency transmission.

Accordingly, we propose *Dynamic Encoding Empowered Spatial Video Content Compression*, as illustrated in Fig. 8. Specifically, we first filter out most background areas using edge detection. Then, we further identify and correct misclassified background areas through RoI partitioning, which is used to identify the RoIs that need encoding. Next, based on these results, we calculate the information saliency map to determine the importance of each pixel within the video frame. Finally, we have developed a dynamic encoding strategy for efficient compression based on saliency information. This approach utilizes low-loss dynamic encoding for important regions of the video, while applying high-loss dynamic encoding to less significant

areas. As a result, we can significantly reduce the amount of data without compromising crucial information.

1) *Edge Detection Based Background Identification*: To alleviate the workload of lightweight edge devices, edge texture information is utilized to separate the background regions leveraging the Prewitt operators [40], represented by $\mathbf{d}_x$ and $\mathbf{d}_y$, i.e., $\mathbf{d}_x = [1, 1, 1; 0, 0, 0; -1, -1, -1]$ and $\mathbf{d}_y = [-1, 0, 1; -1, 0, 1; -1, 0, 1]$. Specifically, for the input frame I_k , the horizontal and vertical gradient matrices $\mathbf{G}_x$ and $\mathbf{G}_y$ are computed as:

$$\mathbf{G}_x = \mathbf{d}_x * I_k, \quad (9)$$

$$\mathbf{G}_y = \mathbf{d}_y * I_k, \quad (10)$$

where $*$ denotes the 2D convolution operator. Let $g_x(i, j)$ and $g_y(i, j)$ represent the elements values of the $\mathbf{G}_x$ and $\mathbf{G}_y$ at pixel (i, j) , respectively. Thus, the element's value of the gradient magnitude matrix at position (i, j) is calculated as

$$g_{xy}(i, j) = \sqrt{g_x(i, j)^2 + g_y(i, j)^2}. \quad (11)$$

If $g_{xy}(i, j) < \lambda_{\text{edge}} \cdot \overline{g_{xy}}(i, j)$, the pixel at position (x, y) is labeled as background, otherwise it is RoI. Finally, the identified RoIs are defined as

$$\mathcal{R}_e = \{(i, j) | g_{xy}(i, j) \geq \lambda_e \cdot \overline{g_{xy}}(i, j)\}, \quad (12)$$

where λ_e is the edge detection threshold, which greatly affects the size of the background and RoI regions.

2) *Region Proposal Partition Based RoI Identification*: Edge detection quickly separates background regions but may misclassify crowd areas by ignoring crowd distribution. To address this and maintain a lightweight design, we reuse crowd detection results to reclassify crowd areas erroneously labeled as background into RoIs.

- 1) *Region Proposal (RP) Box Expansion*: A thorny problem in RP prediction is that the predicted bounding box may not fully cover a person due to motion, leading to inaccurate crowd detection results. To avoid this, we carefully expand the bounding box by $q\%$. Here, this module dynamically configures the value of q for different RP boxes. In particular, it consults the RP prediction confidence level as the indicator to assign different weights to q .
- 2) *RP Batch Partition*: Given a list of predicted RP bounding boxes, a straightforward approach is to encode each box with low-loss quality to the GCS. However, too many partitioning and encoding operations cause high CPU overheads, which are problematic for the limited UAV side. To address this issue, we propose a resource-aware RP partitioning strategy, the core idea of which is to batch process RP boxes. Compared to a single RP box, an RP batch is larger and consists of one or more nearby boxes. Before encoding a frame, we uniformly partition the frame into blocks, each corresponding to an initial RP batch. We calculate the intersection relationship between each RP box and all other RP batches. Given a pair of box a and batch b , their intersection relationship falls into one of the three cases: i) *Full-overlap*. a is completely covered by b , and we conveniently associate them. ii) *No-overlap*. a

is not associated with b . iii) *Partial-overlap*. a intersects with b and at least one other batch. Here, we associate the RP batch with the one most overlapping the RP box.

After all the boxes are associated with an RP batch, we resize each batch to cover all associated boxes fully. Once the previous steps are completed, we can obtain the generated RP batches, denoted as $\mathcal{B}_p$. Recall that d_i and c_i denote the central coordinate and confidence level of the i th bounding box in the frame. Let $c(i, j)$ denote the maximum confidence level of the whole RP batches covering the pixel (i, j) , i.e., $c(i, j) = \max(c_i), \forall d_i \in \mathcal{B}_p$. Thus, the output RoIs are defined as

$$\mathcal{R}_p = \{(i, j) | c(i, j) \geq \lambda_p\}, \quad (13)$$

where λ_p is the preset parameter for the RoI identification.

3) *Information Saliency Map Based Dynamic Encoding*: After obtaining the non-background areas from edge detection and the RoI partitions, we first compute the information saliency map for each frame. These maps are then utilized by an image encoder to encode the video frames dynamically. Note that we use an image encoder rather than a video encoder for the following reasons: after temporal compression, the original video has already undergone keyframe selection. Thus, the video is not composed of continuous frames, and using an image encoder can get better results.

Intuitively, crowd regions within a frame have higher crowd density and are farther away from backgrounds. Therefore, encoding parameters can be categorized according to their importance: image blocks with higher crowd density and farther away from backgrounds have higher encoding quality; conversely, the encoding quality is lower. Specifically, we first use RoI identification results $\mathcal{R}_p$ to complement the background filtering outcomes $\mathcal{R}_e$. Thus, the revised RoI division can be defined as $\mathcal{R}_f = \mathcal{R}_e \cup \mathcal{R}_p$. Moreover, let Θ_k denote the information saliency map of the k th frame, which indicates the information saliency of each pixel for dynamic encoding. Based on the corrected RoI divisions, the element value of Θ_k at pixel (i, j) , represented by $\theta_k(i, j) \in [0, 1]$, is computed as:

$$\theta_k(i, j) = \begin{cases} (1 - \rho_c) + \rho_c \cdot c(i, j), & \text{if } (i, j) \in \mathcal{R}_e \cap \mathcal{R}_p, \\ \rho_c \cdot c(i, j), & \text{if } (i, j) \in \mathcal{R}_f - \mathcal{R}_e, \\ 1 - \rho_c, & \text{if } (i, j) \in \mathcal{R}_f - \mathcal{R}_p, \\ 0, & \text{otherwise,} \end{cases} \quad (14)$$

where $\rho_c \in [0, 1]$ is the weight factor for RoI correction.

Based on the information saliency map, we quantize the encoding quality map to define the actual encoding quality for each pixel, which is computed as

$$\phi_k(i, j) = \begin{cases} \max\{\phi_m, \rho_e \cdot \theta_k(i, j)\}, & \text{if } (i, j) \in \mathcal{R}_f, \\ \phi_0, & \text{otherwise,} \end{cases} \quad (15)$$

where ϕ_m and ϕ_0 are the predefined low-loss encoding quality threshold and lossy encoding quality constant, respectively. ρ_e is the weight coefficient mapping the information saliency to encoding quality, which determines the compression level of detected crowd regions.

TABLE III
STATISTICS OF DATASETS

Dataset	Video resolution	Frame count
SUMCD	1920×1280+	12000+
VisDrone2019	1280×720	8000+
GD	1280×720	2400+
UCF-QNRF	2013×2902	1200+

Finally, the image encoder implements image encoding according to the encoding quality map of each frame. In the above encoding quality setting, regions with almost zero crowd density and also in the background (i.e., regions not belonging to $\mathcal{R}_f$) are assigned lossy encoding quality. Meanwhile, for regions that exist in non-background or have non-zero crowd densities (i.e., regions belonging to $\mathcal{R}_f$), we set them to low-loss compression with the highest compression quality ϕ_m .

V. EVALUATIONS

A. Experimental Setup

Datasets description: The UAV-based crowd surveillance datasets used in this paper include UCF-QNRF [46], Group-detection (GD) [47], Drone Vision Challenge dataset (i.e., VisDrone2019) [48], and the Self-labeled UAV-captured Multi-scenario Crowd Dataset (SUMCD). SUMCD includes various crowd scenarios, such as marathon events, protest activities, and urban squares. All the datasets are compatible with crowd detection tasks and contain over 20,000 UAV-captured video frames with resolutions up to 1280 × 720, covering various scenarios such as crowd riots, large-scale gatherings, and city street parades. The statistics of the datasets are listed in Table III.

Experimental methodology and settings: We have built the S500 UAV (6000 mah battery, 30 min endurance) to capture, process, and transmit surveillance videos. Additionally, we deploy the Jetson Nano (quad-core 64-bit ARM CPU) on the UAV side as the lightweight processing device. In the GCS, a laptop with GPUs serves as the server to communicate with the UAV via WiFi, transferring data in batches to the cloud for crowd-surveillance applications. The cloud server, equipped with 128GB RAM, an NVIDIA 4090 GPU, and an Intel Xeon 8378A CPU, utilizes TensorRT [49] and FFmpeg for deep model inference and video processing, respectively. It employs YOLOv5 as the crowd detection network for both training and detection. Based on our experiments, this server can run YOLOv5 on compressed video frames at an average speed of approximately 60 FPS. Additionally, to improve the YOLOv5 model's adaptability for drone-captured scenarios, we retrained it at the GCS to effectively detect crowded gathering scenes in the videos.

In the field experiments, the flight altitude of the UAV is set to 30 m to guarantee the safety of surveillance tasks and meet the aircraft flight standards; the transmission speed between the UAV and the GCS is about 5 Mbps. Additionally, we mix and sample video frames from different scenes to form the testing set, hence simulating scene switching in UAV-captured

TABLE IV
EXPERIMENTAL PARAMETER SETTINGS

Parameter	Default value	Parameter	Default value
γ	0.5	ε_0	$\frac{\ \mathbf{A}_0\ _1 + \ \mathbf{E}_0\ _1}{40}$
q	25%	λ_e	0.3
λ_p	0.5	ρ_c	0.8
ρ_e	100	ϕ_0	20
ϕ_m	50	bin	25

surveillance videos. The video capturing speed is set to 60 fps. Moreover, various compression algorithms are used to compress and process surveillance videos on the UAV side. Then, the processed data is transmitted to the GCS for crowd-surveillance analysis. The delay and accuracy of each compression algorithm are calculated.

In the experimental parameter setup, we set the default weight factor γ to 0.5 to ensure that the low-rank and sparse parts equally important. Next, we set the convergence threshold ε_0 to $(\|\mathbf{A}_0\|_1 + \|\mathbf{E}_0\|_1)/40$, which adaptively balances the convergence speed and the effectiveness in the temporal video feature compression module. Then, to achieve high accuracy in the spatial compression module, the RP expansion ratio q is set to 0.5, and the block division number is set to bin to 25 by default. Additionally, since the two parameters are important, we analyze the impact of changing the two values on accuracy and latency in the experiment evaluation. Then, the weight factor ρ_c for RoI correction is set to 0.8 for practical efficiency. Furthermore, we manually adjust the other parameters to achieve an approximately optimal balance between accuracy and latency. The experimental parameter settings in this paper are specified in Table IV.

Baseline methods: To evaluate the latency, accuracy, and power of *EagleEye*, we compare it with four representative baseline algorithms. These algorithms adopt different compression strategies and enable comprehensive comparisons from various perspectives.

- *REMIX* [21]: This algorithm compresses video frames based on adaptive image region segmentation in different compression qualities according to the crowd density of each region.
- *CFRC* [50]: Clipped Fixed-RoI Compressing method first clips the UAV-captured video with structural similarity and then compresses frames with fixed RoIs.
- *CVP* [7]: Compressed Video Processing method processes all the UAV-captured video frames with lossy compression.
- *Un-CVP*: The uncompressed video processing method directly transmits the original frames to GCS without compression. Thus, it achieves the highest detection accuracy but incurs the highest overall latency.

Note that we do not use the advanced deep learning-based compression methods as baselines, because the limited AI capabilities of lightweight edge devices make it hard to deploy such algorithms.

Furthermore, to evaluate the performance of each module of our algorithm, two comparison algorithms are used as follows:

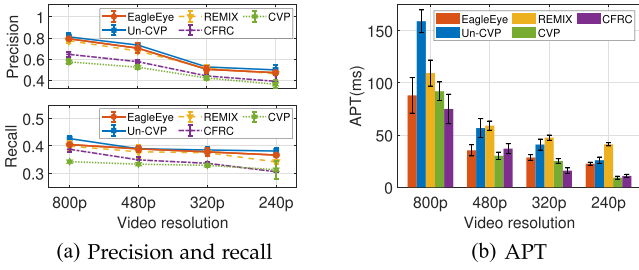


Fig. 9. Performance evaluation with different video resolutions, in terms of precision, recall, and latency.

- *EagleEye w/o SC*: It is a variant of *EagleEye* without the spatial compression module, which compresses the UAV-captured videos via the temporal compression module in this paper.
- *EagleEye w/o TC*: It is a variant of *EagleEye* without the temporal compression module, which compresses the UAV-captured videos by the spatial compression module in this paper.

Evaluation metrics: For the experimental studies, we use three metrics to evaluate the algorithm performance: i) *Average Processing Time (APT)*, representing the sum of the average processing delay at UAV side and the average transmission delay from UAV to GCS per frame; ii) *Precision (P)*, measuring what proportion of positive predictions is actually correct, computed by (16); iii) *Recall (R)*, measuring what proportion of actual positives is identified correctly, computed by (17).

$$P = \frac{TP}{TP + FP}, \quad (16)$$

$$R = \frac{TP}{TP + FN}, \quad (17)$$

where TP denotes that both the label value and the predicted value are true; FP indicates that both the label value and the predicted value are false; FN represents that the label value is false, but the predicted value is true.

B. Experimental Results

Overall performances: We first evaluate the algorithm's performance in different video resolutions. As shown in Fig. 9(a), our algorithm achieves 96.1% detection accuracy and 97.3% recall of Un-CVP. Compared to our algorithm, the detection accuracy of REMIX, CFRC, and CVP is reduced by 1.6%, 16.7%, and 23.3%, respectively. This result indicates that only our algorithm and REMIX consider crowd information during video processing to ensure detection accuracy. Fig. 9(b) shows that our algorithm reduces the APT by 31.4% and 36.2% compared to Un-CVP and REMIX, respectively. The 4% accuracy loss is a reasonable trade-off for the 31.4% latency reduction, as it does not substantially affect the system's effectiveness in identifying high-risk crowd situations. We also evaluate the impact of different crowd-gathering densities on detection performance and processing latency. As shown in Fig. 10(a), our algorithm improves the accuracy by 26.6% and 34.8%, and the recall by 7.9% and 24.0% compared to CVP and CFRC, while achieving

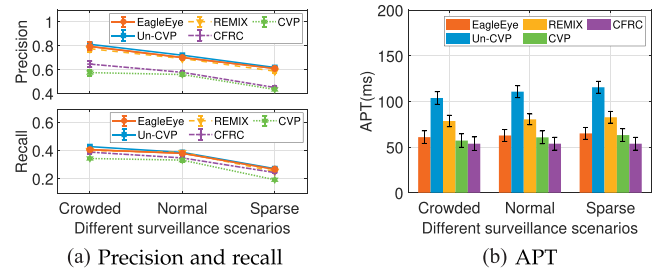


Fig. 10. Performance evaluation in different crowd density scenarios regarding precision, recall, and latency.

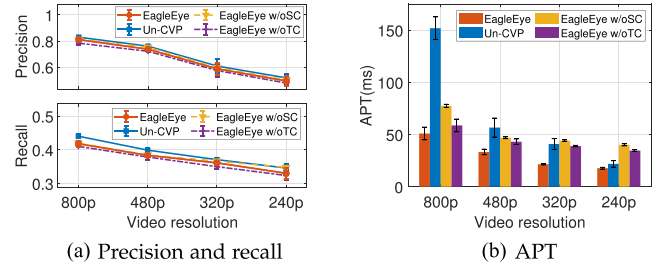


Fig. 11. Ablation studies with different algorithm modules regarding precision, recall, and latency.

crowd detection accuracy and recall close to Un-CVP. Moreover, Fig. 10(b) depicts that our algorithm increases the APT when crowd density increases. In normal and crowded scenes, APT is improved by 3.0% and 6.7% on average compared to sparse scenes, respectively. In all crowd density scenes, our algorithm reduces the APT by 42.8% and 22.0% compared to Un-CVP and REMIX, respectively, with an average reduction of 47.1 ms and 17.7 ms.

Ablation studies on algorithm's module: We conduct ablation studies to highlight the effectiveness of each module. As shown in Fig. 11(a), *EagleEye*, *EagleEye w/o SC*, and *EagleEye w/o TC* achieve 96.7%, 98.0%, and 94.0% of Un-CVP's accuracy, and 96.0%, 97.8%, and 94.0% of Un-CVP's recall, respectively. Fig. 11(b) shows that *EagleEye* reduces the APT of Un-CVP by 43.4%. *EagleEye w/o SC* and *EagleEye w/o TC* reduce the latency at higher resolutions (800p and 480p) by an average of 33.1% and 42.7% compared to Un-CVP, respectively. At lower resolutions (320p and 240p), the delay is increased by 45.6% and 26.6%, respectively.

Evaluation under dynamic scenarios: We evaluate *EagleEye* in real-world dynamic situations using the *HAP Crim Festival of Races 2021* dataset, which includes drone footage of a marathon event captured under various dynamic conditions. First, as shown in Fig. 12(a), when the drone flies at low, middle, high altitudes, the model achieves a precision of 83.9%, 79.8%, and 61.2%, and a recall of 65.2%, 52.0%, and 49.7%, respectively (all under 800p video resolution). Furthermore, Fig. 12(b) shows the model's performance under three challenging dynamic scenarios, i.e., varying drone altitudes, complex crowd changes, and complex environments with dense crowds. In these cases, the precision is 74.2%, 71.8%, and 70.5%, respectively; the recall is 66.6%, 61.4%, and 59.0%, respectively (also under 800p

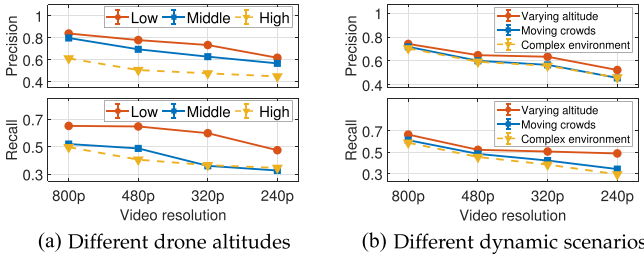


Fig. 12. Performance evaluation with different drone altitudes and dynamic scenarios regarding precision, recall, and latency.

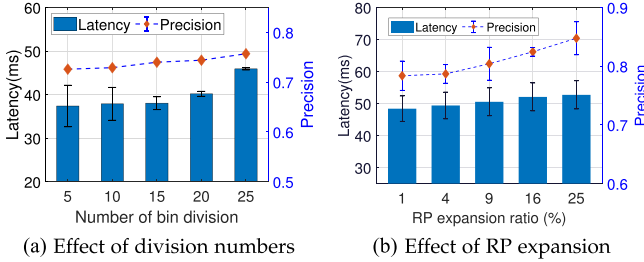


Fig. 13. Parameter evaluation of block division number bin and RP expansion ratio q .

resolution). These results demonstrate that *EagleEye* maintains robust performance across diverse dynamic scenarios.

Algorithm's parameter evaluations: We increase block division number bin from 5 to 25 for experiments to acquire accuracy and APT, then investigate its impact on our algorithm. As shown in Fig. 13(a), the precision is improved by 4.4% and 6.2%, while the APT increases by 8.5 ms on average with the increase of bin . These results indicate that increasing bin can enhance detection performance to some extent but also introduce additional latency. Additionally, we observe the impact of varying RP expansion ratios q on the algorithm's performance; we find that the RP expansion ratio trades off precision and latency. As shown in Fig. 13(b), after increasing the RP expansion ratio from 1% to 25%, the precision improves by 8.2%, while the latency per frame concurrently rises by 4.3 ms. Thus, the block division number bin and the RP expansion ratio q should be configured to balance the accuracy and the delay according to the application scenario.

System latency analysis: We categorize end-to-end latency into two components: drone-side latency and server-side latency. Specifically, drone-side latency includes delays caused by the spatio-temporal video compression algorithm and the transmission of compressed video data to the GCS. Server-side latency consists of 1) the local transmission delay of video data to the cloud server; 2) the delays associated with decoding the video, executing the crowd monitoring algorithm, and displaying the results on the graphical user interface (GUI).

The drone-side latency is illustrated in Fig. 14(a). The results indicate that transmission latency (T_{ts}) and inference latency (T_{inf}) together account for the majority (60%) of the total latency. Additionally, the transmission latency is highly variable due to the instability of the network environment, which leads to fluctuations in bandwidth during data transmission. In contrast,

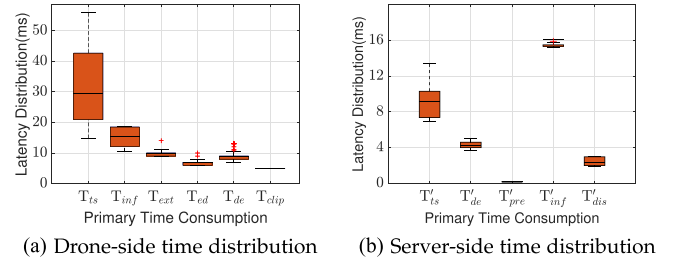


Fig. 14. Analysis of drone-side and server-side time distribution.

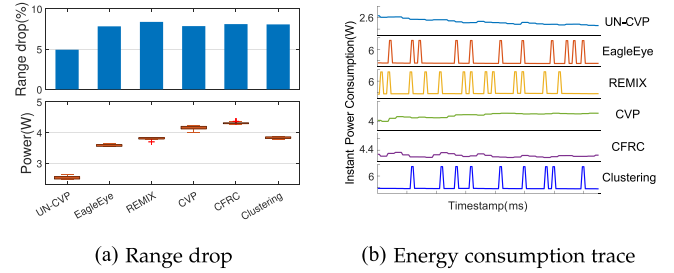


Fig. 15. Analysis of range drop and actual operating power consumption.

the core components of our algorithm—including feature extraction (T_{ext}), edge detection (T_{ed}), dynamic coding (T_{de}), and temporal compression (T_{clip})—exhibit delays of only about 10 ms, representing a smaller proportion of the total latency at 40%. Then, as shown in Fig. 14(b), on the server-side, transmission (T'_{ts}) and inference (T'_{inf}) still contribute to the majority (78.9%) of the total latency. The remaining parts—including decoding (T'_{de}), preprocessing (T'_{pre}), and display (T'_{dis})—only contribute approximately 6.75 ms of delay in total (21.1%).

These results verify that *EagleEye* employs a lightweight modular design, enabling crowd surveillance to execute quickly on a high-performance cloud. As a result, the system achieves a low end-to-end latency, ensuring timely and efficient processing in urban surveillance scenarios.

Algorithm's power and efficiency: In addition, we evaluate the power consumption, endurance (range drop), energy consumption trace, and GPU/RAM utilization of various algorithms on real edge devices. To obtain precise measurements, we run a dedicated power monitoring program during practical experiments. Fig. 15(a) and (b) demonstrate that our algorithm exhibits stable and efficient performance across different scenarios. Specifically, the average range drop caused by our algorithm is only 7.8%, with an average power consumption of 3.58 W (maximum 3.65 W, minimum 3.53 W). Compared to REMIX, CFRC, and Clustering, it reduces power consumption by 7.2%, 3.7%, and 3.2%, respectively. Then, Fig. 16(a) demonstrates that the RAM usage of *EagleEye* is consistent with REMIX, CVP, CFRC, and Clustering, while the GPU usage is reduced by up to 25.1% and 38.1% compared to REMIX and Clustering, respectively. Furthermore, we compare our algorithm with the conference version of this paper (i.e., *Clustering*) on video compression ratio. As shown in Fig. 16(b), our algorithm's

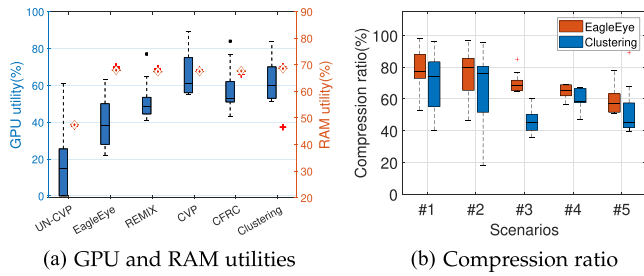


Fig. 16. Analysis of actual GPU utility, RAM utility, and compression ratio.

compression ratio is 11.0% higher than clustering on average, indicating the enhanced robustness of our approach.

Trade-off comparison analysis: To illustrate the trade-offs among latency, accuracy, and power in *EagleEye*, we present pair-wise scatter plots in Fig. 17(a), (b), and (c). All experimental results are obtained under the same settings, i.e., using the same 800p-resolution video and processed on the Jetson Nano. The results show that *EagleEye* achieves 97.6% of the precision of Un-CVP, while reducing the APT by 44.7%. In terms of power consumption, *EagleEye* reduces it by 6.2%, 16.8%, and 13.7% compared to REMIX, CVP, and CFRC, respectively. These results demonstrate that *EagleEye* achieves a well-balanced trade-off across all three dimensions compared to baseline methods. In addition, we conduct an ablation study to analyze the trade-off between compression ratio and precision, as shown in Fig. 17(d). Specifically, we compare five methods using the same video segments and compression configurations. The results indicate that *EagleEye* achieves the highest compression ratio of 93.5% while maintaining high detection precision, outperforming *EagleEye* w/o SC (68.8%), *EagleEye* w/o TC (79.1%), and CVP (85.6%). This result demonstrates that the temporal and spatial modules play a critical role in video compression and high-precision detection.

System practical applications: To verify the practical effect of *EagleEye*, we evaluate *EagleEye* in four typical surveillance applications, including violence detection, face recognition, crowd counting, and content decomposition. First, the results are shown in Fig. 18 for the violence detection tasks within large-scale gathering scenarios. The results indicate that our algorithm can effectively identify violent behavior in densely populated areas, which performs similarly to Un-CVP, but CVP fails to recognize such behavior. Next, we conduct the validation experiment in face recognition. Fig. 19 shows that *EagleEye* can reach almost the same accuracy as Un-CVP, whereas CVP misidentifies on multiple face groups. Then, in crowd counting tasks with four large-scale gathering scenes (i.e., crowded city street, downtown park, plaza square, and marathons), as shown in Fig. 20, *EagleEye* outperforms CVP in four large-scale gathering scenarios, with similar detection results like Un-CVP in all scenarios. Next, for the content decomposition task, Fig. 21 shows that *EagleEye* can effectively separate inefficient content in most scenarios. However, its performance partially degrades in specific scenarios with drastic view changes or high crowd density, such as in the 4th image of Fig. 21. At last, we present an example comparing the original and compressed frames in

Fig. 22. The compressed frame shows that the road area is significantly compressed, while the crowd remains almost unchanged after applying *EagleEye*. Therefore, the result indicates that *EagleEye* can effectively compress the image's inefficient part and significantly reduce the frame's memory size.

In summary, the experimental results presented above further validate the effectiveness of our algorithm for real-time, high-precision video compression in crowd surveillance. In comparison with baseline methods, our algorithm achieves remarkable improvement in reducing video transmission latency by 31.4% while maintaining the detection accuracy loss within 4%. Also, the performance of our algorithm is stable and robust, making it adaptable to crowd supervision in various surveillance scenarios.

C. System Implementation

We develop a UAV-assisted urban crowd surveillance prototype system to validate the effectiveness and feasibility of *EagleEye*. As shown in Fig. 23, we utilize an S500 UAV [35] equipped with a lightweight processing device (i.e., Jetson Nano) as the edge node. The UAV is also equipped with a motion camera, flight control, and a wireless transceiver to capture and stream HD video. Additionally, a GPU-equipped laptop serves as the ground station, communicating with the UAV in real-time via WiFi.

The prototype system executes crowd surveillance tasks in three steps: i) *Deployment*: The ground station is placed near crowded areas in urban environments, and the UAV patrols above the crowd-gathering areas. ii) *Video Capturing*: The UAVs fly around to capture videos of gathering crowds, while the edge devices on the UAVs perform spatio-temporal low-loss video compression and transmit the compressed videos to the ground station with low latency. iii) *Crowd Surveillance*: The ground station conducts crowd surveillance applications (e.g., crowd counting and density analysis) on the UAV-captured videos. The results are then uploaded to a cloud server for visualization. As shown in Fig. 23, the system interface displays real-time streaming video, detection results of crowds (e.g., crowd density information and specific person recognition results), and the UAVs' flight trajectories and status. This interface provides a comprehensive overview of the surveillance information.

VI. DISCUSSIONS & FUTURE WORK

Impact of adverse weather on system performance: Our work aims to develop a large-scale crowd surveillance system based on cameras. It leverages visual data for dense crowd detection and behavior analysis. However, such vision-based approaches are highly susceptible to adverse weather conditions (e.g., low-light, fog, and rain), which would lead to blurred images, increased occlusion, and missing visual features. These issues can significantly degrade detection performance. To address these negative influences, we can add additional monitor sensors (such as infrared sensors and millimeter-wave radar) to enhance the system's robustness under low-visibility conditions. Furthermore, from an algorithmic perspective, we will explore data augmentation techniques tailored for adverse weather scenarios to improve the system's generalization capability [51]. In

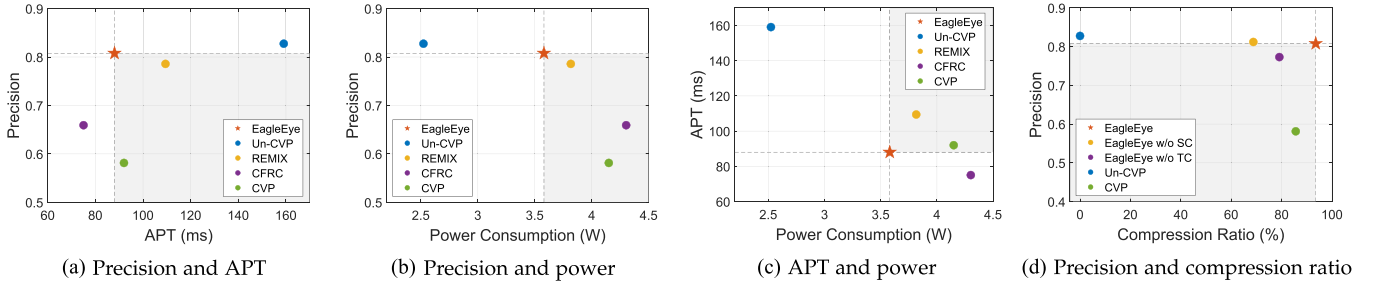


Fig. 17. Analysis of trade-offs among: (a) precision vs. APT, (b) precision vs. power consumption, (c) APT vs. power consumption, and (d) precision vs. compression ratio.

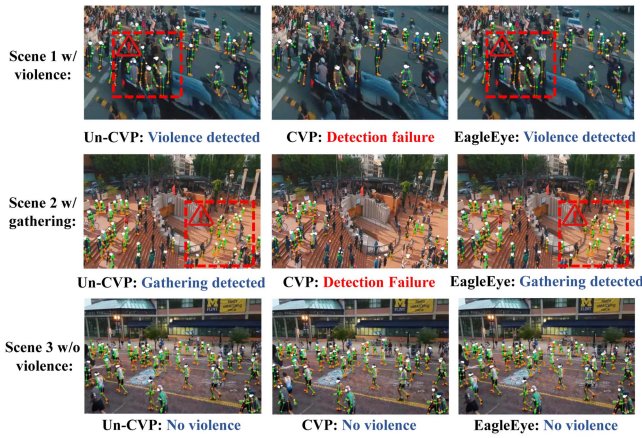


Fig. 18. Illustration of violence detection in four large-scale gatherings scenarios. Note that the first two columns realistically contain crowd violence and gathering behavior, and the last column is a normal scene without violence.



Fig. 19. Illustration of 4 face recognition examples from 5760 face recognition results, including multiple scenarios (such as outdoor sports, indoor meetings, and dim light).

the future, we will further study how to improve *EagleEye*'s performance in adverse weather by adding additional monitor sensors and exploring data augmentation techniques.

Adaptive parameter optimization: Determining the optimal parameter configuration for a specific environment is critical in improving the adaptability and robustness of *EagleEye*. Therefore, we use an adaptive parameter tuning process to find the best parameter settings in the experiments as follows. First, we select a representative scenario (e.g., a crowded street environment). Next, we define candidate values for key parameters (for instance, $\lambda_e \in \{0.2, 0.25, 0.3, 0.35, 0.4\}$ and $\rho_e \in \{0.7, 0.8, 0.9\}$), which are then used to construct a 5×3

parameter grid. Moreover, we employ a grid search strategy [52], where each combination of parameters (i.e., each grid cell) is systematically tested. The algorithm is trained and tested under the target scenario using each combination, while the performance is recorded. Finally, we select the parameter pair that yields the highest F1 score in this scenario.

Security of Video Transmission: We use Wi-Fi communication for drone video transmission, which makes drones vulnerable to network attacks. To enhance the security of video transmission while meeting lightweight requirements, we can adopt the Elliptic Curve Cryptography algorithm [53] as follows. Specifically, when the edge device compresses the video data, it divides the compressed video into several small data blocks. And then, a temporary key is generated using the public key obtained from the ground station in advance, which encrypts each data block. The encrypted data blocks are then transmitted over the wireless network. Once the ground station receives the data, it uses the corresponding private key to decrypt the temporary key and access the video data. Also, we can improve the encryption and authentication of Wi-Fi communication by implementing the WPA3 protocol [54]. WPA3 enhances password protection and encryption techniques, making it more difficult for attackers to gain network access through brute-force password cracking. In the future, we will further study how to enhance the security of video transmission.

Efficient compression by jointly using *EagleEye* and H.265. H.265 is widely used as the video compression standards that efficiently compress and encode video data [55]. As H.265 is complementary to *EagleEye*, we consider combining our algorithm with it to further reduce transmission latency without reducing crowd surveillance accuracy. First, we apply an existing spatio-temporal compression algorithm to process the raw video frames. Then, the processed video is input into the H.265 encoder, which records only the parts of each frame that have changed and uses entropy coding (e.g., CABAC) for encoding [56]. Next, the encoded bitstream is packaged into a bitstream format and transmitted via Wi-Fi to the GCS. Finally, the H.265 decoder parses the bitstream and reconstructs the video on the GCS side. The reconstructed video is then used for crowd surveillance tasks. In the future, we will study how to integrate H.265 into *EagleEye* to achieve efficient compression.

Practical impact of accuracy loss: According to dynamic analysis [57], individuals can move freely without the risk of

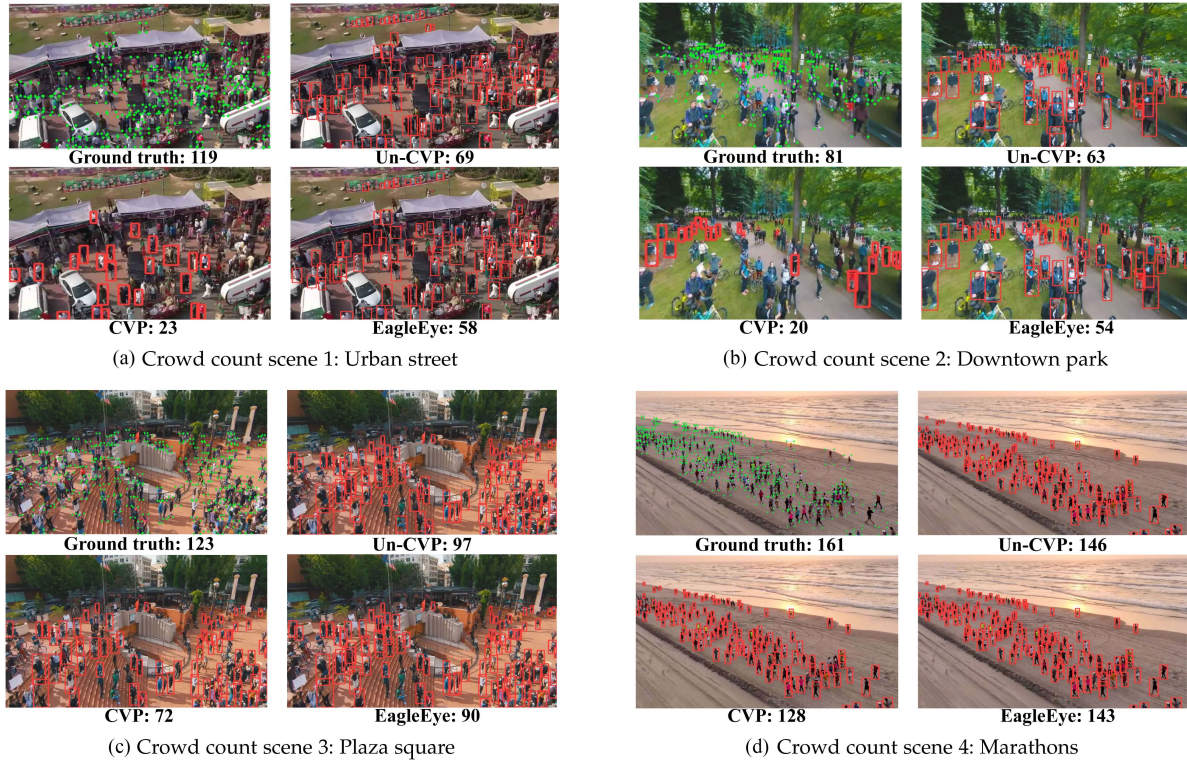


Fig. 20. Illustration of crowd detection results in four large-scale gatherings scenarios, including urban street, election rally, plaza square, and marathons.



Fig. 21. Efficient and inefficient content extraction by *EagleEye* from pedestrian flows and static background areas.

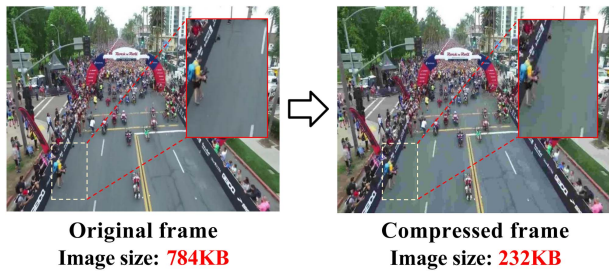


Fig. 22. Comparison of original (on the left) and compressed frames (on the right).

dangerous events when crowd density is between 1 to 2 people per m^2 . The probability of risk remains low at densities of 3 to 4 people per m^2 . However, when density reaches 5 people per m^2 , individual movement becomes restricted, increasing the risk of

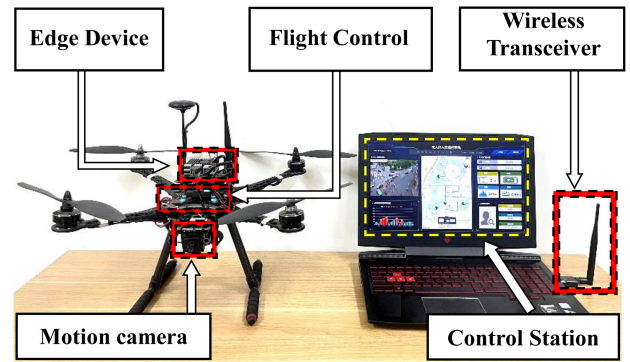


Fig. 23. UAV-based real-time crowd surveillance prototype system. The UAV has a lightweight device, a motion camera, and flight control to capture and process videos.

incidents. Once density exceeds 6 people per m^2 , individuals are forced to crowd together and lose the ability to move freely, which significantly raises the risk of serious accidents (such as stampedes and asphyxiation). For example, during the Itaewon stampede incident of 2022, an extreme crowd density of 9 to 10 people per m^2 was recorded, far surpassing the safe limit. Based on this analysis, the spatio-temporal compression algorithm of *EagleEye* shows that when compared to uncompressed video, the error in estimated crowd density is about 0.36 to 0.4 people per m^2 . Since a density exceeding 5 people per m^2 indicates a high risk, this measurement error does not significantly affect *EagleEye*'s ability to identify high-density crowd areas. In the

future, we will apply *EagleEye* in real-world applications to evaluate its actual effectiveness.

VII. CONCLUSION

This paper proposes *EagleEye*, a *low-power, high-precision*, and *low-latency* crowd surveillance system empowered by edge-assisted UAVs, at the heart of which lies a spatio-temporal video feature compression algorithm tailored for capability-limited UAV-side edge. By integrating Compressive Sensing theory for temporal compression and a dynamic encoding scheme based on the information saliency map for spatial compression, *EagleEye* achieves low processing and transmission latency while little loss of detection accuracy under low power consumption.

ACKNOWLEDGMENT

A preliminary version of this work appeared in IEEE ICA3PP 2023 [1].

REFERENCES

- [1] Q. Zhang, D. Wan, H. Chen, L. Cheng, J. Chen, and C. Xiang, "Spatio-temporal fusion based low-loss video compression algorithm for UAVs with limited processing capability," in *Proc. Algorithms Architectures Parallel Process.: 23rd Int. Conf.*, Tianjin, China, 2023, pp. 191–211.
- [2] "South Korean authorities say they had no guidelines for halloween crowds, as families grieve 156 victims," 2022. [Online]. Available: <https://edition.cnn.com/2022/10/31/asia/seoul-itaewon-halloween-mourning-memorial-intl-hnk/index.html>
- [3] "Hajj stampede: What we know so far," 2015. [Online]. Available: <https://www.bbc.com/news/world-middle-east-34357952>
- [4] A. Singh, D. Patil, and S. Omkar, "Eye in the sky: Real-time drone surveillance system (DSS) for violent individuals identification using scatternet hybrid deep learning network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshop*, Salt Lake City, USA, 2018, pp. 1629–1637.
- [5] J. Yi, S. Choi, and Y. Lee, "EagleEye: Wearable camera-based person identification in crowded urban spaces," in *Proc. 26th Annu. Int. Conf. Mobile Comput. Netw.*, London, U.K., 2020, pp. 1–14.
- [6] J. Liu, C. Gao, D. Meng, and A. G. Hauptmann, "DecideNet: Counting varying density crowds through attention guided detection and density estimation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, 2018, pp. 5197–5206.
- [7] Y. Jiang, Y. Miao, B. Alzahrani, A. Barnawi, R. Alotaibi, and L. Hu, "Ultra large-scale crowd monitoring system architecture and design issues," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10356–10366, Jul. 2021.
- [8] D. Clifton, "Patrol," in *Hospitality Security*. Boca Raton, FL, USA: CRC Press, 2022, pp. 119–178.
- [9] T. Yu, Y. Takaku, Y. Kaieda, and K. Sakaguchi, "Design and PoC implementation of mmWave-based offloading-enabled UAV surveillance system," *IEEE Open J. Veh. Technol.*, vol. 2, pp. 436–447, 2021.
- [10] "CCTV: Too many cameras useless, warns surveillance watchdog tony porter," 2019. [Online]. Available: <https://www.bbc.com/news/uk-30978995>
- [11] B. Jiang, J. Yang, and H. Song, "Protecting privacy from aerial photography: State of the art, opportunities, and challenges," in *Proc. IEEE Conf. Comput. Commun. Workshops*, Toronto, ON, Canada, 2020, pp. 799–804.
- [12] C. Xiang et al., "Reusing delivery drones for urban crowdsensing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2972–2988, May 2023.
- [13] Y. Wan, Y. Zhong, A. Ma, and L. Zhang, "An accurate UAV 3-D path planning method for disaster emergency response based on an improved multiobjective swarm intelligence algorithm," *IEEE Trans. Cybern.*, vol. 53, no. 4, pp. 2658–2671, Apr. 2023.
- [14] H. Bai, S. Wen, and S.-H. Gary Chan, "Crowd counting on images with scale variation and isolated clusters," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop*, Seoul Korea, 2019, pp. 18–27.
- [15] G. Castellano, C. Castiello, M. Cianciotta, C. Mencar, and G. Vessio, "Multi-view convolutional network for crowd counting in drone-captured images," in *Proc. Comput. Vis. 2020 Workshop*, Glasgow, U.K., 2020, pp. 588–603.
- [16] Y. Yang, G. Li, Z. Wu, L. Su, Q. Huang, and N. Sebe, "Reverse perspective network for perspective-aware object counting," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Seattle, WA, USA, 2020, pp. 4374–4383.
- [17] K. Rezaee, S. J. Mousavirad, M. R. Khosravi, M. K. Moghimi, and M. Heidari, "An autonomous UAV-assisted distance-aware crowd sensing platform using deep shufflenet transfer learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 9404–9413, Jul. 2022.
- [18] L. Xiao, Y. Ding, J. Huang, S. Liu, Y. Tang, and H. Dai, "UAV anti-jamming video transmissions with QoE guarantee: A reinforcement learning-based approach," *IEEE Trans. Commun.*, vol. 69, no. 9, pp. 5933–5947, Sep. 2021.
- [19] J. Sun, B. Li, Y. Jiang, and C.-Y. Wen, "A camera-based target detection and positioning UAV system for search and rescue (SAR) purposes," *Sensors*, vol. 16, no. 11, 2016, Art. no. 1778.
- [20] L. Liu, H. Li, and M. Gruteser, "Edge assisted real-time object detection for mobile augmented reality," in *Proc. 25th Annu. Int. Conf. Mobile Comput. Netw.*, Los Cabos, Mexico, 2019, pp. 1–16.
- [21] S. Jiang, Z. Lin, Y. Li, Y. Shu, and Y. Liu, "Flexible high-resolution object detection on edge devices with tunable latency," in *Proc. 27th Annu. Int. Conf. Mobile Comput. Netw.*, New Orleans, Louisiana, 2021, pp. 559–572.
- [22] X. Lv, Q. Wang, C. Yu, and H. Jin, "A feedback-driven DNN inference acceleration system for edge-assisted video analytics," *IEEE Trans. Comput.*, vol. 72, no. 10, pp. 2902–2912, Oct. 2023.
- [23] T. Tan and G. Cao, "Deep learning on mobile devices through neural processing units and edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1209–1218.
- [24] W. Zhang et al., "ELF: Accelerate high-resolution mobile deep vision with content-aware parallel offloading," in *Proc. 27th Annu. Int. Conf. Mobile Comput. Netw.*, New Orleans, Louisiana, 2021, pp. 201–214.
- [25] S. MahmoudZadeh, A. Yazdani, A. Elmi, A. Abbasi, and P. Ghanooni, "Exploiting a fleet of UAVs for monitoring and data acquisition of a distributed sensor network," *Neural Comput. Appl.*, vol. 34, no. 7, pp. 5041–5054, 2022.
- [26] G. Castellano, E. Cotardo, C. Mencar, and G. Vessio, "Density-based clustering with fully-convolutional networks for crowd flow detection from drones," *Neurocomputing*, vol. 526, pp. 169–179, 2023.
- [27] J. Guo and C. Peng, "Towards live video analytics with on-drone deeper-yet-compatible compression," 2021, *arXiv:2111.06263*.
- [28] L. Xu, Q. Yang, M. Qin, W. Wu, and K. Kwak, "Collaborative human recognition with lightweight models in drone-based search and rescue operations," *IEEE Trans. Veh. Technol.*, vol. 73, no. 2, pp. 1765–1776, Feb. 2024.
- [29] A. Chowdhery and M. Chiang, "Model predictive compression for drone video analytics," in *Proc. IEEE Int. Conf. Sens., Commun. Netw.*, Hong Kong, China, 2018, pp. 1–5.
- [30] X. Zhang, L. Wang, X. Wu, L. Xu, and A. Fei, "Energy-efficient computation offloading and data compression for UAV-mounted MEC networks," in *Proc. IEEE Glob. Commun. Conf.*, Kuala Lumpur, Malaysia, 2023, pp. 6904–6909.
- [31] Y. Li et al., "Reducto: On-camera filtering for resource-efficient real-time video analytics," in *Proc. Annu. Conf. ACM Special Int. Group Data Commun. Appl., Technol., Architectures, Protoc. Comput. Commun.*, New York, NY, USA, 2020, pp. 359–376.
- [32] S. Lu, X. Yuan, and W. Shi, "Edge compression: An integrated framework for compressive imaging processing on CAVs," in *Proc. IEEE/ACM Symp. Edge Comput.*, San Jose, CA, USA, 2020, pp. 125–138.
- [33] K. Deng et al., "Geryon: Edge assisted real-time and robust object detection on drones via mmWave radar and camera fusion," in *Proc. ACM Interactive, Mobile, Wearable Ubiquitous Technol.*, vol. 6, no. 3, pp. 1–27, 2022.
- [34] "Nvidia jetson nano," 2022. [Online]. Available: <https://www.nvidia.cn/autonomous-machines/embedded-systems/jetson-nano/>
- [35] "S500 v2 kit," 2022. [Online]. Available: <http://www.holybro.com/product/pixhawk4-s500-v2-kit/>
- [36] "Jetson benchmarks," 2023. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-benchmarks>
- [37] "Mobilenet-yolo," 2021. [Online]. Available: <https://github.com/dogquiqui/MobileNet-Yolo>
- [38] C. Chen, J. Cai, W. Lin, and G. Shi, "Incremental low-rank and sparse decomposition for compressing videos captured by fixed cameras," *J. Vis. Commun. Image Representation*, vol. 26, pp. 338–348, 2015.
- [39] "YOLOv4-tiny," 2022. [Online]. Available: <https://github.com/AlexeyAB/darknet>
- [40] R. C. Gonzalez, *Digital Image Processing*. London, U.K.: Pearson, 2009.

- [41] D. C. Lay, S. R. Lay, and J. J. McDonald, *Linear Algebra and its Applications*. 5th ed. London, U.K.: Pearson, 2016.
- [42] J. Wright, A. Ganesh, S. Rao, Y. Peng, and Y. Ma, "Robust principal component analysis: Exact recovery of corrupted low-rank matrices via convex optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2009, pp. 2080–2088.
- [43] F. Nie, J. Yuan, and H. Huang, "Optimal mean robust principal component analysis," in *Proc. 31st Int. Conf. Int. Conf. Mach. Learn.*, Beijing, China, 2014, pp. 1062–1070.
- [44] X. Zhang, W. Lin, R. Xiong, X. Liu, S. Ma, and W. Gao, "Low-rank decomposition-based restoration of compressed images via adaptive noise estimation," *IEEE Trans. Image Process.*, vol. 25, no. 9, pp. 4158–4171, Sep. 2016.
- [45] C. Xiang et al., "Edge computing-empowered large-scale traffic data recovery leveraging low-rank theory," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 1–13, Oct.–Dec. 2020.
- [46] H. Idrees et al., "Composition loss for counting, density map estimation and localization in dense crowds," in *Proc. Comput. Vis. 2018: 15th Eur. Conf.*, Munich, Germany, 2018, pp. 532–546.
- [47] F. Solera, S. Calderara, and R. Cucchiara, "Socially constrained structural learning for groups detection in crowd," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 5, pp. 995–1008, May 2015.
- [48] D. Du et al., "Visdrone-Det2019: The vision meets drone object detection in image challenge results," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop*, Seoul Korea, 2019, pp. 213–226.
- [49] "Nvidia tensorrt," 2022. [Online]. Available: <https://developer.nvidia.com/tensorrt>
- [50] Z. Wang, E. P. Simoncelli, and A. C. Bovik, "Multiscale structural similarity for image quality assessment," in *Proc. 37th Asilomar Conf. Signals, Syst. Comput.*, Pacific Grove, CA, USA, 2003, pp. 1398–1402.
- [51] T. Kumar, R. Brennan, A. Mileo, and M. Bendeche, "Image data augmentation approaches: A comprehensive survey and future directions," *IEEE Access*, vol. 12, pp. 187536–187571, 2024.
- [52] D. M. Belete and M. D. H., "Grid search in hyperparameter optimization of machine learning models for prediction of Hiv/Aids test results," *Int. J. Comput. Appl.*, vol. 44, no. 9, pp. 875–886, 2022.
- [53] S. Ullah, J. Zheng, N. Din, M. T. Hussain, F. Ullah, and M. Yousaf, "Elliptic curve cryptography; applications, challenges, recent advances, and future trends: A comprehensive survey," *Comput. Sci. Rev.*, vol. 47, 2023, Art. no. 100530.
- [54] G. Sagers, "WPA3: The greatest security protocol that may never be," in *Proc. IEEE Int. Conf. Comput. Sci. Comput. Intell.*, Las Vegas, NV, USA, 2021, pp. 1360–1364.
- [55] Z.-N. Li, M. S. Drew, and J. Liu, *Modern Video Coding Standards: H.264, H.265, and H.266*. Berlin, Germany: Springer, 2021, pp. 423–478.
- [56] D. Karwowski, "Precise probability estimation of symbols in VVC CABAC entropy encoder," *IEEE Access*, vol. 9, pp. 65361–65368, 2021.
- [57] M. Tschechne, "Nine per square meter," *Max Planck Research*, no. 2, pp. 32–37, 2022.



Chaocan Xiang received the BS and PhD degrees in computer science and engineering from the Nanjing Institute of Communication Engineering, China, in 2009 and 2014, respectively. He is a full professor with the College of Computer Science, Chongqing University, China. He studied at the Real-Time Computing Lab with the University of Michigan, Ann Arbor, in 2017. His current research interests include mobile networks and IoT. He has published more than 40 research papers in important conferences and journals, such as ACM UbiComp, IEEE INFO-

COM, *IEEE/ACM Transactions on Networking*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Parallel and Distributed Systems*, and *IEEE Transactions on Intelligent Transportation Systems*.



Zhenghan Li received the BE degree in software engineering from the Dalian University of Technology, Dalian, China, in 2022, and the MS degree in computer science from Ritsumeikan University, Osaka, Japan, in 2024. He is currently working toward the PhD degree with Chongqing University, Chongqing, China. His research interests include wireless sensor networks and mobile computing.



Qianyuan Zhang received the BE degree in communication engineering from the Chongqing University of Posts and Telecommunications, Chongqing, China, in 2021, and the MS degree from Chongqing University, Chongqing, in 2024. His research interests include drones, crowd-sensing networks and IoT.



Xuanguo Wu received the PhD degree in computer software and theory from the University of Science and Technology of China, Hefei, China, in 2013. From 2018 to 2019, he was a visiting scholar with the Department of Computer Science and Engineering, the Hong Kong University of Science and Technology. He is currently a full professor with the School of Computer Science and Technology, Anhui University of Technology, Ma'anshan, China. His research interests include Internet of Things, network security, and privacy protection.



Xiao Zheng received the BS degree from Anhui University, China, in 1997, the MS degree from the Zhejiang University of Science and Technology, China, in 2003, and the PhD degree in computer science and technology from Southeast University, China, in 2014. He is currently the professor with the School of Computer Science and Technology, Anhui University of Technology, Anhui, China.



Yulan Guo (Senior Member, IEEE) received the BE and PhD degrees from the National University of Defense Technology (NUDT), in 2008 and 2015, respectively. He has authored more than 150 articles at highly referred journals and conferences. His research interests lie in 3D vision, low-level vision, and machine learning. He served as an associate editor for *IEEE Transactions on Image Processing*, the *Visual Computer*, *IET Computer Vision*, *IET Image Processing*, and *Computers & Graphics*. He also served as an area chair for CVPR 2023/2021, ICCV 2021, and ACM Multimedia 2021. He organized several workshops, challenges, and tutorials in prestigious conferences such as CVPR 2019/2022/2023, ICCV 2021/2023, and ECCV 2022. He is a Senior Member of ACM.